

C++ & Software Engineering Project

Project Specifications and Guidelines

Université Bourgogne Europe

Academic Year 2025–2026

Contents

1	Context & Objectives	2
2	Core Architecture Constraints	2
3	Hints and suggested milestones	2
3.1	Milestone 0: Preparation & Architecture Design (Prior to Coding)	2
3.2	Milestone 1: Mesh Parsing & Graph Construction	3
3.3	Milestone 2: 3D Rendering & User Interaction	3
3.4	Milestone 3: Core Topological Editing Engine	3
3.5	Milestone 4: Software Architecture & Design Patterns	3
4	Grading & Evaluation Scheme	3
5	Grading & Evaluation Scheme	4

1 Context & Objectives

This project serves a key pedagogical purpose: it requires authentic, low-level C++ software engineering—rather than high-level AI prompt generation—to solve real-world problems. By exploring graph and mesh representations, it establishes natural connections with robotics and spatial reasoning while ensuring full structural coherence with the module’s Midterm and Final written examinations.

Graphs form the foundation of numerous software engineering and computer science domains, including pathfinding, network analysis, and physical simulations. In many fields, complex spatial surfaces are modeled using **triangular meshes**. A triangular mesh is a specialized graph embedded in $\mathbb{R}^3$, where nodes represent 3D positions and edges dictate surface connectivity.

The goal of this project is to design an interactive 3D Mesh Editor in modern C++. You will build a topological graph engine capable of importing geometric files, rendering them, and executing real-time topological modifications (e.g., vertex insertion, edge splitting, face removal) and optionally geometric modifications (vertex displacements/transforms).

Recommended References & Learning Materials

To familiarize yourself with the Half-Edge data structure and its topological operations, you are strongly encouraged to consult the following academic references:

- **UC Berkeley CS184 — Computer Graphics and Imaging:**
Half-Edge Mesh Data Structure Guide.
<https://cs184.eecs.berkeley.edu/sp19/article/15/the-half-edge-data-structure>

2 Core Architecture Constraints

Your implementation must adhere strictly to the canonical **Half-Edge Data Structure** using 4 minimal pointers per half-edge:

- **vertex:** Pointer to the single **target (destination)** vertex of the directed edge.
- **face:** Pointer to the single bounded face lying on the left-hand side.
- **next:** Pointer to the next half-edge bounding the same face in **counter-clockwise (CCW)** order.
- **twin:** Pointer to the opposite half-edge on the adjacent face running in **clockwise** direction.

3 Hints and suggested milestones

While you are entirely free to manage your project schedule as you see fit, past experience shows that students all too often neglect the early preparation phase, wait until the final weeks, and end up rushing the implementation under extreme stress. The following milestones are specifically designed as a roadmap to help you distribute your workload smoothly across the semester. Above all, keep in mind that you have other courses, projects, and examinations to manage simultaneously: plan ahead and organize your time as effectively as possible!

3.1 Milestone 0: Preparation & Architecture Design (Prior to Coding)

Before writing any implementation code, you must plan your software architecture and project organization:

- **UML Class Diagram:** Design a comprehensive class diagram detailing the components of your topological engine (**Vertex**, **Face**, **Edge**, **HalfEdge**, **Mesh**, etc.), including explicit relationships, access modifiers, and memory ownership strategies.
- **Development Roadmap:** Establish a detailed step-by-step roadmap outlining feature implementation order, team member roles/responsibilities (if applicable), and milestone deadlines. Development usually also include a B-plan strategy to avoid dead-ends and time wastes.

Do not forget to create your Github repository, to invite Yohan Fougerolle.

3.2 Milestone 1: Mesh Parsing & Graph Construction

- Implement an importer/exporter for standard 3D file formats (`.obj` or `.ply ASCII`). Feel free to support additional formats.
- Reconstruct the full Half-Edge data structure from raw vertex/face lists.

3.3 Milestone 2: 3D Rendering & User Interaction

- Render the 3D mesh using a graphics library of your choice (OpenGL, SFML, GLFW, etc.).
- Support wireframe and shaded surface rendering modes.
- Implement an interface allowing users to select vertices, faces, or edges.

3.4 Milestone 3: Core Topological Editing Engine

Implement the following pure topological operations while ensuring complete structural integrity (all 4 pointers per half-edge must remain consistent):

1. **Vertex & Face Instantiation:** Connect 3 existing or newly added vertices to create a new face, allocating and linking the corresponding half-edges.
2. **Edge Split:** Insert a new vertex along an edge, dividing adjacent faces and rebuilding local half-edge connectivity.
3. **Edge Flip:** Flip the shared edge between two adjacent triangles (V_0, V_1, V_2) and (V_1, V_0, V_3) to connect V_2 and V_3 , modifying local pointer cycles without changing vertex coordinates.
4. **Entity Removal:** Delete a vertex or face and repair local connectivity (handling boundary half-edges properly).

3.5 Milestone 4: Software Architecture & Design Patterns

- **Undo/Redo System:** Implement the **Command Pattern** to allow reversing and re-applying editing operations (`Ctrl+Z` / `Ctrl+Y`).
- **Clean Architecture:** Demonstrate Object-Oriented compliance and clean separation of concerns between data, rendering, and interaction.

4 Grading & Evaluation Scheme

This project accounts for **30% of your overall semester grade**. Your final score will be evaluated based on the following breakdown:

5 Grading & Evaluation Scheme

This project accounts for **30% of your overall semester grade**. Your final score will be evaluated based on the following breakdown:

Evaluation Criteria	Description & Weight
Consistency & Engagement (20%)	Regular work throughout the semester, evidenced by a steady GitHub commit history and active participation/questions during lab sessions and classes.
Core Low-Level Implementation (25%)	Quality, correctness, and memory safety of base C++ classes and low-level Half-Edge topological algorithms.
Documentation & Report (25%)	Clarity, thoroughness, and technical quality of the user guide, class diagrams, and architectural documentation.
Final Software Quality (20%)	Overall polish, stability, user interface design, and functional completeness of the final application.
Originality & Personal Initiatives (10%)	Innovative features, advanced extensions, or clever algorithmic/UI enhancements beyond the minimal specifications.

At the end of the project, your team must submit the following items:

- **GitHub Repository Link:** A link to a private/public GitHub repository containing your full C++ source code. The commit history must clearly reflect continuous, regular work from all team members across the entire semester.
- **Technical Documentation & User Guide:** A comprehensive document (included in the repository as a PDF report written in $\text{\LaTeX}$) containing:
 1. A user manual with visual illustrations/screenshots demonstrating how to run and use your software.
 2. Detailed explanations of your software architecture, including class diagrams, design choices, and memory ownership models.